PAGINATION

A Practical Guide for Mobile & Backend Engineers



KRISHANU NANDAN

Pagination: A Practical Guide for Mobile & Backend Engineers

Pagination is one of the most critical yet underestimated parts of system design. While it often starts as a simple UI concern, pagination directly affects **performance, scalability, user experience, backend cost, and data consistency**.

This e-book explains pagination from theory to real-world systems used by companies like **Instagram, Zomato, and Amazon**, with a strong focus on **Android engineering**.

TABLE OF CONTENTS

1. **Pagination**
2. **Types of Pagination**
3. **Real-World Case Studies**
4. **System Design Diagrams (End-to-End)**
5. **Sequence Diagrams**
6. **Database Index Design**
7. **Android Paging 3 State Machine**

1. Pagination (Theory)

What Is Pagination?

Pagination is the process of breaking a large dataset into smaller chunks (pages) and loading them incrementally instead of fetching everything at once.

Example:

- Instead of loading 100,000 products
- Load 20–50 items per request

Pagination is used in:

- Product lists
- Feeds and timelines
- Search results
- Chat history
- Transaction logs

Why Do We Need Pagination?

1. **Performance**
    - Faster initial load
    - Lower memory usage
    - Smooth scrolling (no jank)
2. **Network Efficiency**
    - Smaller payloads
    - Faster retries

- Better mobile data usage

3. **User Experience**
    - Infinite scrolling
    - Progressive loading
    - Faster perceived performance
4. **Backend Scalability**
    - Reduced database load
    - Predictable query performance
    - Lower infrastructure cost

At scale, pagination is not optional — it's mandatory.

---

2. Types of Pagination (Offset vs Cursor)

2.1 Offset-Based Pagination

How It Works

```
GET /products?page=3&limit=20
```

Database logic:

```
OFFSET = (page - 1) * limit
```

Pros

- Simple to implement
- Easy to understand
- Supports “Go to page X”

Cons

- Slow on large datasets
- OFFSET causes row scanning
- Data inconsistency when new records are inserted
- Poor choice for infinite scrolling

Best Use Cases

- Admin dashboards
- Small datasets
- Static data
- Back-office tools

2.2 Cursor-Based Pagination

How It Works

Instead of a page number, the client sends a **cursor** referencing the last item.

```
GET /feed?limit=20&cursor=post_102
```

Database logic:

```
WHERE id > cursor

LIMIT 20
```

Pros

- Extremely fast
- Stable results
- Scales well
- Perfect for infinite scroll

Cons

- Cannot jump to arbitrary pages
- Slightly more complex
- Requires indexed, immutable fields

Best Use Cases

- Feeds and timelines
- Infinite scrolling
- Large, frequently updated datasets

3. Real-World Case Studies

3.1 Instagram – Feed Pagination

Problem

- Billions of posts
- Constant new inserts
- Infinite scrolling
- No duplicates or missing items

Strategy Used

- Cursor-based pagination
- Cursor = createdAt + postId

Example:

```
GET /feed?before=1705669200
```

Database logic:

```
WHERE (created_at, id) < (cursorCreatedAt, cursorId)
ORDER BY created_at DESC, id DESC
LIMIT 20
```

Key Takeaway

Infinite scrolling + real-time inserts require cursor-based pagination.

3.2 Zomato – Restaurant Discovery

Problem

- Location-based search
- Dynamic filters (rating, distance)
- Sorting by relevance or distance

Strategy Used

- Cursor pagination with sort key

Cursor example:

```
cursor = "3.4km|restaurant_981"
```

Database logic:

```
WHERE (distance, id) > (cursorDistance, cursorId)
ORDER BY distance ASC, id ASC
LIMIT 20
```

Key Takeaway

When sorting or filtering changes dynamically, cursor pagination is safer than offset.

3.3 Amazon – Hybrid Pagination

Amazon uses **different pagination strategies** based on context.

- User-facing search → Hybrid (page number + internal cursor token)
- Admin tools → Offset pagination

User sees:

```
?page=3
```

Backend uses:

```
nextToken = "abc123"
```

Key Takeaway

UI pagination and backend pagination do not need to be the same.

---

4. System Design Diagrams (End-to-End)

High-Level Pagination Flow

```
Android UI

↓

PagingDataAdapter

↓

PagingSource

↓

Repository
```

```
↓

Backend API

↓

Service Layer

↓

Database (Indexed)
```

Pagination decisions affect **every layer** of the system.

---

5. Sequence Diagrams

5.1 Infinite Scroll Sequence

```
User scrolls

↓

PagingDataAdapter requests data

↓

PagingSource.load()

↓

Repository.fetch(cursor)

↓

API receives cursor

↓
```

```
Database query using index

↓

Results + nextCursor returned

↓

UI renders new items
```

5.2 Pull-to-Refresh Sequence

```
User pulls to refresh

↓

PagingSource.invalidate()

↓

New PagingSource created

↓

First page requested (no cursor)

↓

Fresh data rendered
```

This ensures new items appear only after refresh.

---

6. Database Index Design

6.1 Offset Pagination (Inefficient)

```
Scan rows
↓
Skip OFFSET rows
↓
Return LIMIT rows
```

Problems:

- Full or partial table scans
- Poor performance at scale

6.2 Cursor Pagination (Efficient)

```
Index seek to cursor
↓
Read next N rows
```

Index example:

```
INDEX (id)
```

6.3 Feed Pagination (Best Practice)

Composite index:

```
INDEX (created_at DESC, id DESC)
```

Query:

```
WHERE (created_at, id) < (?, ?)

ORDER BY created_at DESC, id DESC

LIMIT 20
```

Important:

- Index order must match ORDER BY

---

7. Android Paging 3 State Machine

Paging 3 Load States

```
Idle

↓

Loading

↓

Loaded

↓

Append / Prepend
```

```
↓

End of Pagination
```

Error & Retry Flow

```
Loading

↓

Error

↓

Retry

↓

Loading
```

UI Mapping

Paging State	UI Behavior
Refresh Loading	Full-screen loader
Append Loading	Footer spinner
Error	Retry button

End reached	No footer

---

8. Final Takeaways

- Pagination is a **system design problem**, not just a UI feature
- Offset pagination does not scale for infinite scrolling
- Cursor-based pagination is the industry standard for feeds
- Database indexes are as important as API design
- Android Paging 3 works best with cursor pagination
- Always design pagination **end-to-end** (UI → DB)

Final Thought

> Pagination is not about loading data — it's about controlling **order, consistency, and scale** in distributed systems.

Thank You.

Follow: - [Krishanu Nandan]